

Rapport scientifique — Projet de fin de module de Deep Learning

Conception, implémentation, comparaison et analyse critique de modèles de deep learning pour données tabulaires, images et séquences

Par : RAIS Yasser

Professeur : Mr. Mossab

Année universitaire 2025–2026

Table des matières

Rapport scientifique — Projet de fin de module de Deep Learning.....	1
Liste des tableaux	4
1. Introduction.....	5
2. Objectifs.....	5
3. Méthodologie générale	6
4. Partie I — MLP et ingénierie PyTorch	6
Classification tabulaire sur <i>Breast Cancer Wisconsin</i>	6
4.1 Cadre théorique.....	6
4.2 Préparation des données.....	6
4.3 Implémentation	6
4.4 Résultats expérimentaux	7
4.5 Interprétation et analyse critique	8
5. Partie II — CNN et vision par ordinateur	9
Classification d'images sur <i>Fashion-MNIST</i>	9
5.1 Pourquoi un CNN plutôt qu'un MLP ?	9
5.2 Calculs dimensionnels et implémentations « maison »	9
5.3 Modèle et expériences	9
5.4 Interprétation et analyse critique	12
6. Partie III — RNN, LSTM, GRU et Seq2Seq.....	13
Modélisation de séquences et traduction EN→FR sur <i>fra-eng</i>	13
6.1 Modèle de langage et perplexité	13
6.2 Préparation des données.....	13
6.3 Comparaison RNN / LSTM / GRU	13
6.4 BPTT et gradient clipping	14
6.5 Système Seq2Seq et décodage	15
Table 6 - Exemples de traductions (glouton vs beam search).	16
6.6 Interprétation et analyse critique	16
7. Synthèse transversale	17
8.Limites	18
9.Conclusion.....	18

Liste des figures

Figure 1 - Comparaison des stratégies d'initialisation (MLP).	7
Figure 2 - Matrice de confusion sur le test (MLP).	8
Figure 3 - Échantillons du jeu de données Fashion-MNIST.	9
Figure 4 - Effet des choix architecturaux (CNN).	10
Figure 5 - Cartes de caractéristiques de la première convolution.	10
Figure 6 - Comparaison MLP vs CNN (même dataset).	11
Figure 7 - Matrice de confusion (CNN).	11
Figure 8 - Perplexité de validation : RNN vs LSTM vs GRU.	14
Figure 9 - Effet du gradient clipping sur la stabilité de la BPTT.	15
Figure 10 - Apprentissage du système Seq2Seq (EN→FR).	15

Liste des tableaux

Table 1 - Jeux de données utilisés par partie	5
Table 2 - Métriques d'évaluation sur le test (MLP).	7
Table 3 - Étude architecturale du CNN (un facteur à la fois).	10
Table 4 - Comparaison RNN / LSTM / GRU (modèle de langage).	14
Table 5 - Scores BLEU selon la stratégie de décodage.	16
Table 6 - Exemples de traductions (glouton vs beam search).	16
Table 7 - Biais inductifs des trois familles d'architectures.	18

1. Introduction

Le deep learning ne propose pas une architecture universelle, mais une famille d'architectures dont chacune encode une hypothèse structurelle (*biais inductif*) adaptée à un type de données. Ce projet démontre, par l'expérimentation, que le choix de l'architecture doit suivre la nature des données : tabulaires, images ou séquences. Trois études complémentaires sont menées sous PyTorch, chacune sur un jeu de données réel :

Partie	Famille	Données	Dataset
I	MLP	Tabulaire	Breast Cancer Wisconsin
II	CNN	Image	Fashion-MNIST
III	RNN/LSTM/GRU/Seq2Seq	Séquence (texte)	fra-eng (Tatoeba)

Table 1 - Jeux de données utilisés par partie

Chaque partie articule théorie, implémentation, expériences comparatives et analyse critique, conformément au cahier des charges.

2. Objectifs

- Maîtriser la construction de modèles PyTorch (`nn.Module`, `nn.Sequential`, classes personnalisées), la gestion des paramètres, l'initialisation, la sauvegarde/rechargement et l'usage CPU/GPU.
- Comprendre et implémenter les opérations convolutives (corrélation croisée 2D, pooling), étudier l'effet des choix architecturaux et interpréter les représentations internes.
- Modéliser des séquences (modèle de langage, perplexité, BPTT, gradient clipping), comparer RNN/LSTM/GRU et construire un système de traduction Seq2Seq évalué par BLEU avec décodage glouton et *beam search*.
- Relier, dans une synthèse transversale, ces trois familles à la structure des données.

3. Méthodologie générale

Principe d'ingénierie : aucun paramètre codé en dur. Chaque notebook débute par une cellule de configuration (@dataclass Config) centralisant chemins, hyperparamètres et graine aléatoire ; tout le code y fait référence. La reproductibilité est assurée par une graine fixée (random, numpy, torch) et la portabilité CPU/GPU par une sélection automatique du *device*.

Protocole expérimental. Découpage systématique apprentissage / validation / test, normalisation ajustée uniquement sur l'apprentissage (pas de fuite d'information), sélection du meilleur modèle sur la validation, évaluation finale sur le test. Les comparaisons procèdent « toutes choses égales par ailleurs » (un facteur varié à la fois).

Environnement de reproduction des figures de ce rapport. PyTorch 2.x (CPU). Pour rester exécutable sans GPU, les budgets sont volontairement modestes : Fashion-MNIST est sous-échantillonné (12 000 images d'entraînement), le corpus fra-eng limité à 6 000 paires courtes, avec un nombre d'époques réduit. Ces budgets sont réglables depuis la Config ; les augmenter rapproche des performances de référence de la littérature. Les figures ci-dessous sont réellement produites par le code du projet (script artifacts/figures/).

4. Partie I — MLP et ingénierie PyTorch

Classification tabulaire sur *Breast Cancer Wisconsin*

4.1 Cadre théorique

Un MLP est une composition de couches linéaires $y = xW^T + b$ et de non-linéarités (ReLU). La propagation avant calcule la sortie puis la perte ; la rétropropagation (`loss.backward()`) calcule $\partial L / \partial \theta$ par la règle de la chaîne, et l'optimiseur met à jour $\theta \leftarrow \theta - \eta \partial L / \partial \theta$. Les paramètres apprenables sont suivis par *autograd* ; le `state_dict()` (dictionnaire {nom : tenseur}) est le format standard de sauvegarde ; le *device* impose que modèle et données résident au même endroit (CPU ou GPU). Les données tabulaires n'ayant ni voisinage spatial ni temporalité, le MLP — qui traite tous les attributs symétriquement — constitue le biais inductif minimal adéquat.

4.2 Préparation des données

569 tumeurs, 30 variables morphologiques. Nettoyage (suppression d'id et de la colonne vide), encodage de la cible (M=malin $\rightarrow$ 1, B=bénin $\rightarrow$ 0), découpage stratifié 64 % / 16 % / 20 %, standardisation ajustée sur l'apprentissage.

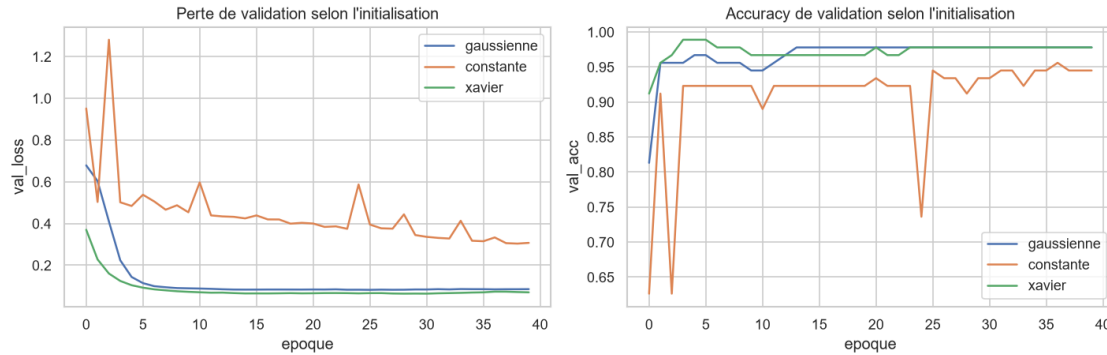
4.3 Implémentation

Deux versions du même réseau (topologie [64, 32], ReLU, dropout 0,1) : l'une bâtie avec `nn.Sequential`, l'autre comme classe personnalisée (`nn.Module` à forward explicite). Les

paramètres sont inspectés via `named_parameters()` et `state_dict()` ; le gradient, `None` avant rétropropagation, apparaît après `backward()`. Le meilleur modèle est sauvegardé puis rechargé depuis son `state_dict`, et la cohérence de `device` est vérifiée.

4.4 Résultats expérimentaux

Initialisation. Trois stratégies comparées (gaussienne, constante, Xavier) :



Comparaison des initialisations

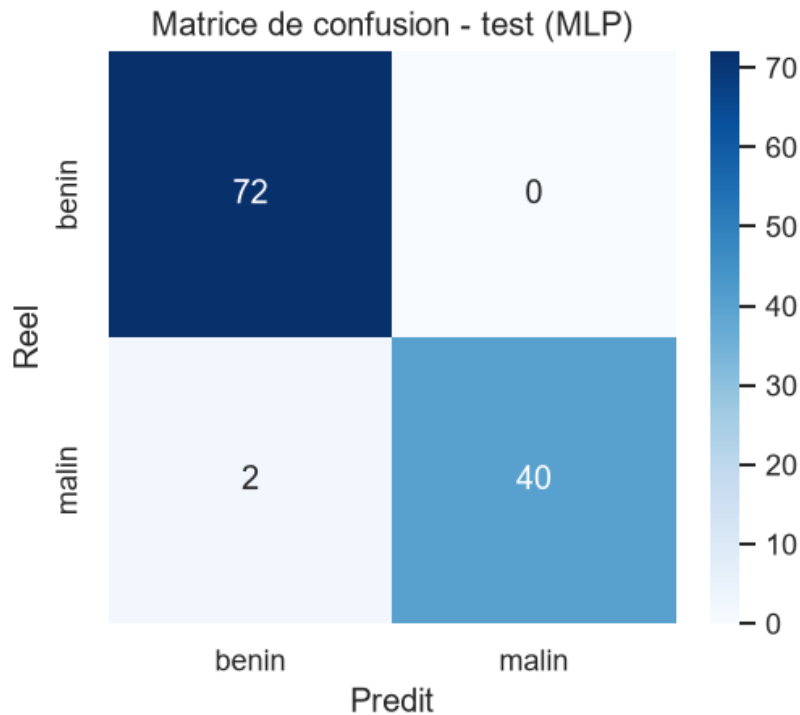
Figure 1 - Comparaison des stratégies d'initialisation (MLP).

L'initialisation constante stagne (val_acc finale 0,945) : tous les neurones d'une couche partageant le même poids reçoivent le même gradient et restent identiques — c'est le problème de symétrie. Les initialisations gaussienne et Xavier brisent cette symétrie et atteignent 0,978.

Évaluation sur le test (meilleur modèle, ~4 130 paramètres) :

Table 2 - Métriques d'évaluation sur le test (MLP).

Métrique	Valeur
Accuracy	0,9825
Précision (malin)	1,0000
Rappel (malin)	0,9524
F1-score	0,9756
ROC-AUC	0,9947



Matrice de confusion MLP

Figure 2 - Matrice de confusion sur le test (MLP)

4.5 Interprétation et analyse critique

Le MLP atteint un F1 de 0,976 et une AUC de 0,995 : *Breast Cancer Wisconsin* est quasi linéairement séparable après normalisation, ses descripteurs étant fortement discriminants. La précision de 1,0 signifie aucun faux positif ; le rappel de 0,952 indique quelques faux négatifs — dans un contexte clinique, c'est la grandeur à surveiller (ne pas manquer une tumeur maligne). Les versions Sequential et personnalisée, à topologie égale, sont équivalentes : le choix relève du style d'ingénierie.

Question de synthèse — Partie I. *Dans quelle mesure un MLP bien paramétré est-il pertinent pour la classification tabulaire, et quelles sont ses limites ?* — Un MLP normalisé et régularisé est très pertinent ici : frontières non linéaires sans hypothèse de voisinage, $F1 > 0,97$. Ses limites tiennent à la structure statistique des données : absence de biais dédié au tabulaire (interactions de seuil, variables catégorielles mieux gérées par les arbres), sensibilité au sur-apprentissage sur petit échantillon (569 exemples → dropout, *weight decay*, *early stopping* nécessaires), interprétabilité moindre. Son avantage s'érode dès que les données deviennent hétérogènes ou peu nombreuses, où le gradient boosting reste compétitif.

5. Partie II — CNN et vision par ordinateur

Classification d'images sur *Fashion-MNIST*

5.1 Pourquoi un CNN plutôt qu'un MLP ?

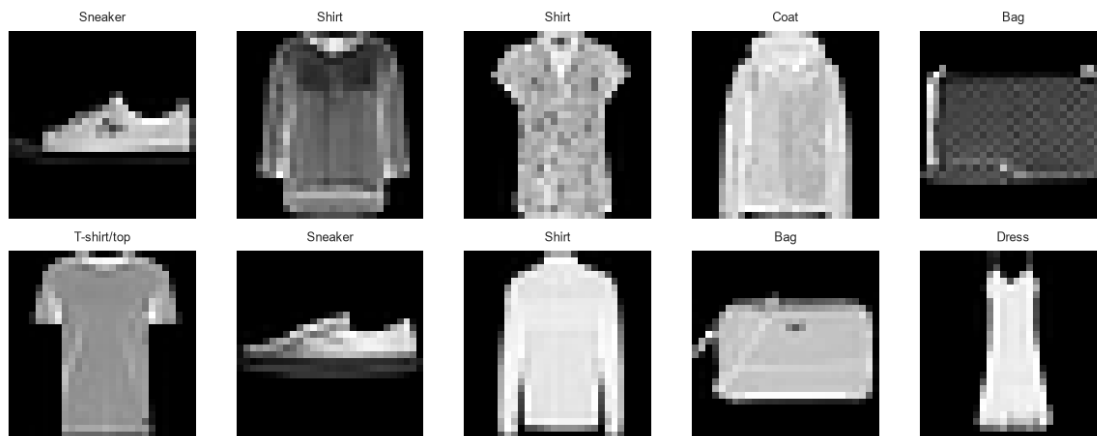
Aplatir une image 28×28 en 784 entrées détruit la localité 2D et expose le nombre de paramètres. Les CNN encodent trois idées fondatrices : la localité (champs réceptifs restreints), le partage des poids (un noyau glissant $\rightarrow$ invariance par translation et chute du nombre de paramètres) et la hiérarchie des représentations (bords $\rightarrow$ motifs $\rightarrow$ objets).

5.2 Calculs dimensionnels et implémentations « maison »

La taille de sortie d'une convolution vaut $H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$; ex. $28 \rightarrow 24$ pour $k = 5, p = 0, s = 1$, et $28 \rightarrow 14$ pour $k = 3, p = 1, s = 2$. La corrélation croisée 2D, le max-pooling et l'average-pooling ont été reprogrammés avec des boucles explicites et validés contre `F.conv2d`, `F.max_pool2d` et `F.avg_pool2d` (résultats identiques).

5.3 Modèle et expériences

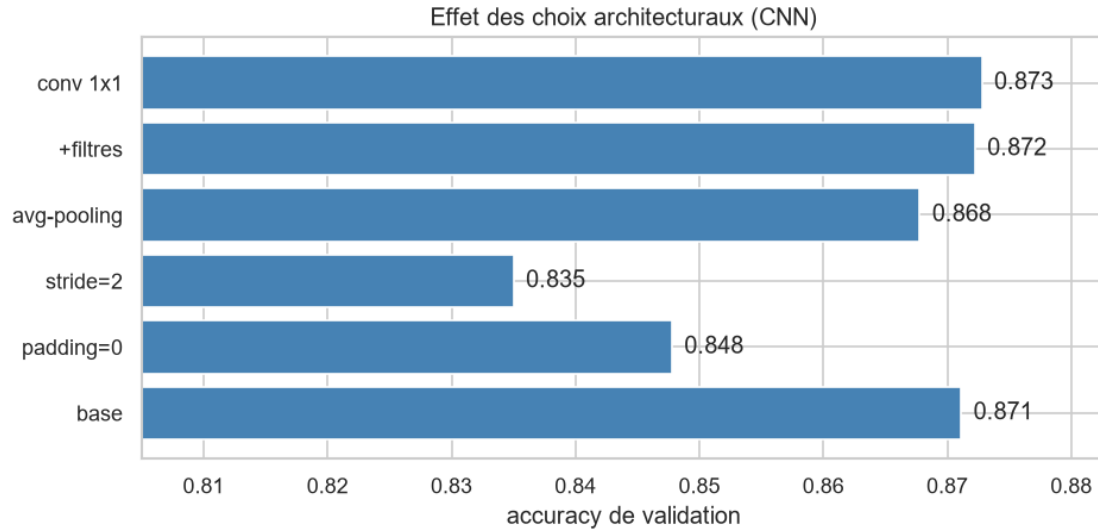
Architecture inspirée de LeNet, améliorée par BatchNorm (variante autorisée), taille aplatie déduite dynamiquement (aucun calcul codé en dur). Aperçu du jeu de données :



Échantillons Fashion-MNIST

Figure 3 - Échantillons du jeu de données *Fashion-MNIST*.

Étude « un facteur à la fois » autour d'une base (padding 2, stride 1, max-pooling, 6/16 filtres) :



Effet des choix architecturaux

Figure 4 - Effet des choix architecturaux (CNN).

Table 3 - Étude architecturale du CNN (un facteur à la fois).

Variante	Accuracy val.	Paramètres
base	0,8711	107 830
padding = 0	0,8478	44 470
stride = 2	0,8350	21 430
average-pooling	0,8678	107 830
plus de filtres (16/32)	0,8722	212 638
conv 1×1	0,8728	107 872

Cartes de caractéristiques de la première convolution :

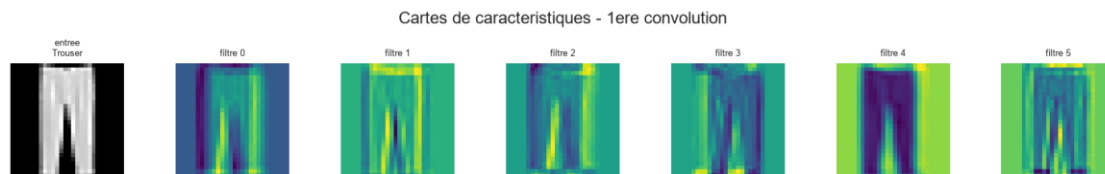


Figure 5 - Cartes de caractéristiques de la première convolution.

Comparaison MLP vs CNN sur le même jeu de données :

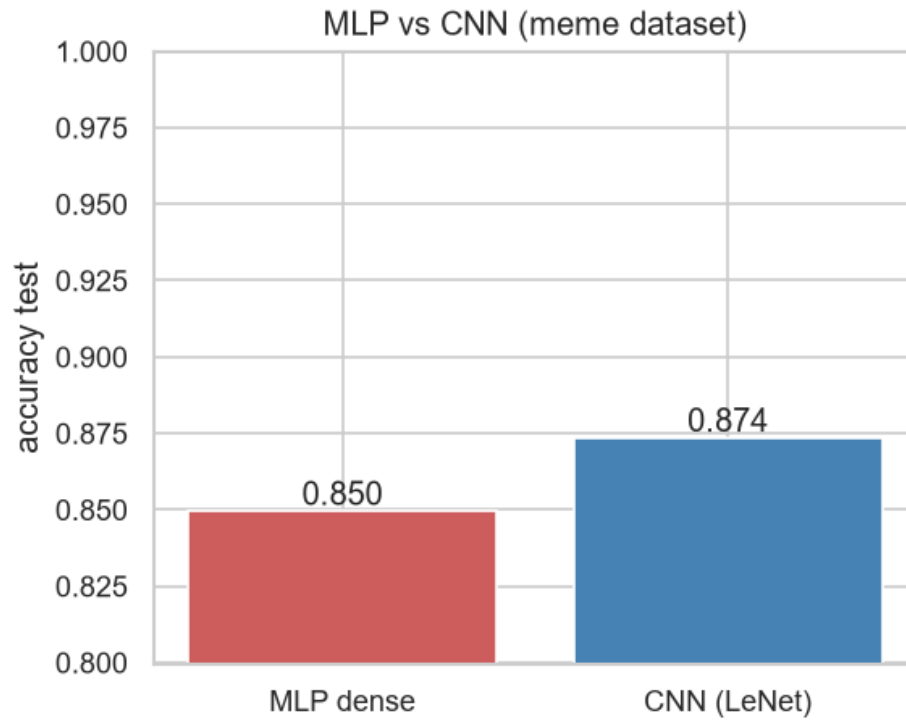
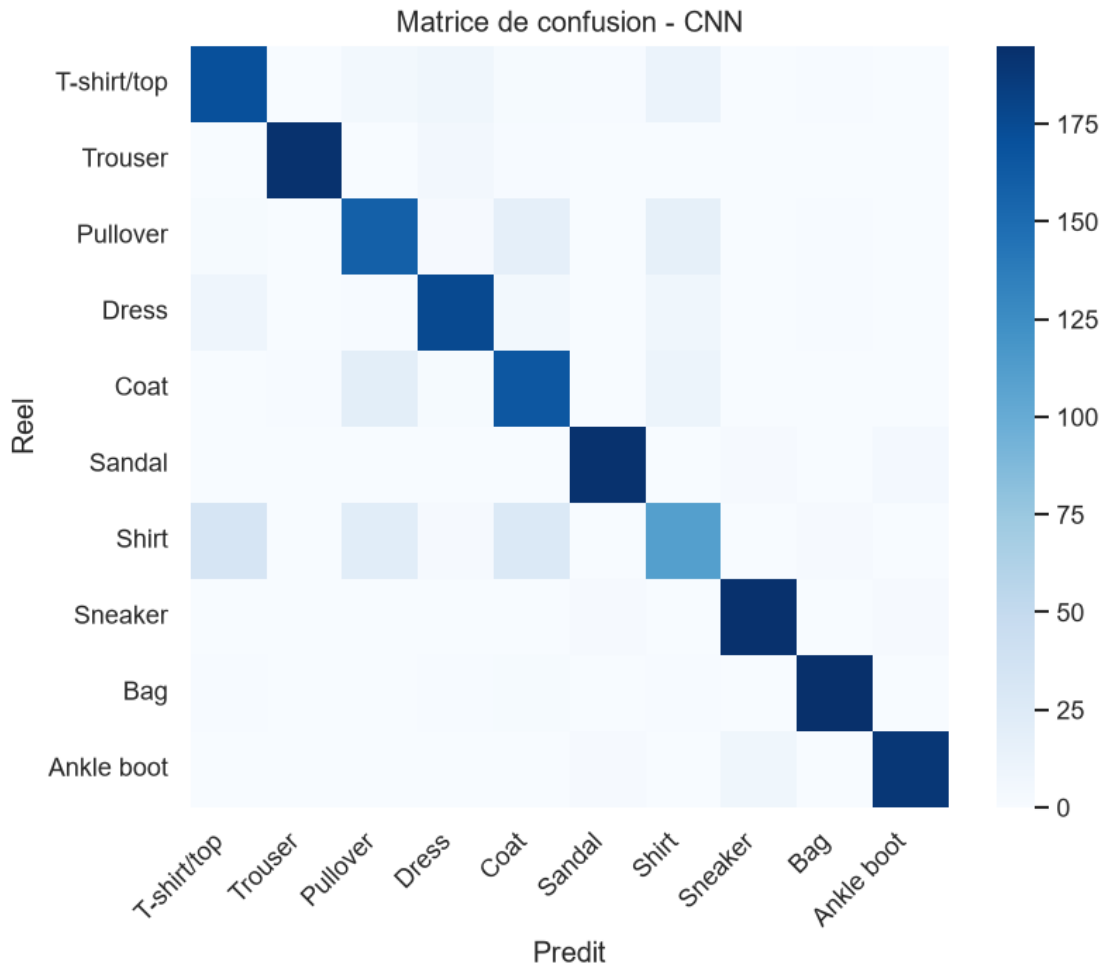


Figure 6 - Comparaison MLP vs CNN (même dataset).

Figure 7 - Matrice de confusion (CNN).



5.4 Interprétation et analyse critique

- CNN > MLP : le CNN atteint 0,874 contre 0,850 pour le MLP, avec deux fois moins de paramètres (≈ 108 k contre ≈ 235 k). Le partage des poids est donc à la fois plus performant et plus efficace — illustration directe de la valeur d'un bon biais inductif.
- padding = 0 rogne les bords et réduit la carte → légère perte (0,848).
- stride = 2 sous-échantillonne dès la convolution : calcul réduit (21 k paramètres) mais résolution dégradée → chute à 0,835.
- max- vs average-pooling : le max conserve les activations saillantes (0,871 vs 0,868).
- plus de filtres / conv 1×1 : gains marginaux ici (capacité déjà suffisante) ; la conv 1×1 recombine les canaux à coût quasi nul.
- Les cartes de caractéristiques montrent des filtres sensibles à différentes orientations de bords — matérialisation de la hiérarchie des représentations.
- Les confusions résiduelles concernent des classes visuellement proches (*Shirt / T-shirt / Pullover / Coat*).

Question de synthèse — Partie II :

Pourquoi un CNN est-il plus pertinent qu'un MLP, et comment padding/stride/pooling/profondeur influencent-ils les performances ? — Le CNN encode la structure spatiale que le MLP détruit ; nos mesures le confirment (meilleure accuracy à paramètres moindres). Les choix dimensionnels agissent de façon quantifiable via $H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$: le padding préserve l'information de bord, le stride arbitre coût/résolution, le pooling apporte une invariance locale, la profondeur et le nombre de filtres augmentent capacité et champ réceptif au prix d'un coût et d'un risque de sur-apprentissage accrus.

6. Partie III — RNN, LSTM, GRU et Seq2Seq

Modélisation de séquences et traduction EN→FR sur *fra-eng*

6.1 Modèle de langage et perplexité

Un modèle de langage factorise $P(x_1, \dots, x_T) = \prod_t P(x_t | x_{<t})$ (règle de chaîne) : il suffit de prédire le prochain token à partir du passé, résumé par l'état caché récurrent. La qualité se mesure par la perplexité $PPL = \exp(-\frac{1}{T} \sum_t \log P(x_t | x_{<t}))$ — exponentielle de l'entropie croisée moyenne ; plus basse = meilleure.

6.2 Préparation des données

Corpus parallèle Tatoeba : nettoyage, tokenisation, vocabulaires source et cible avec tokens spéciaux (<pad>, <bos>, <eos>, <unk>), troncature/padding et masquage de la perte sur les positions de remplissage, mini-lots tensoriels.

6.3 Comparaison RNN / LSTM / GRU

Trois cellules récurrentes derrière une interface commune, perte masquée :

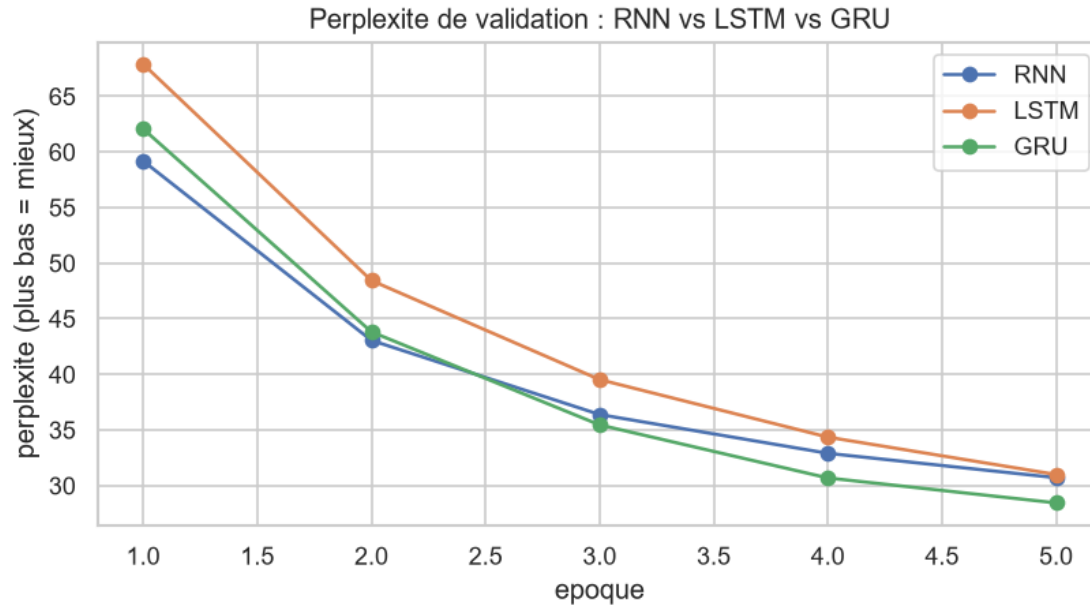


Figure 8 - Perplexité de validation : RNN vs LSTM vs GRU

Modèle	Perplexité (val.)	Paramètres	Temps/époque
RNN simple	30,70	472 178	0,94 s
LSTM	30,99	558 962	1,00 s
GRU	28,44	530 034	1,19 s

Table 4 - Comparaison RNN / LSTM / GRU (modèle de langage).

Le GRU obtient la meilleure perplexité. Le RNN simple est le plus léger et rapide. À ce petit budget (6 000 paires, 5 époques), le LSTM n'exprime pas encore son avantage (plus de paramètres à entraîner) : son intérêt se révèle sur des séquences plus longues et davantage de données.

6.4 BPTT et gradient clipping

La BPTT dépile le réseau dans le temps ; le gradient traverse une chaîne de produits jacobiens, d'où des gradients évanescents ou explosifs. En provoquant l'instabilité (RNN tanh, SGD à fort *learning rate*) :

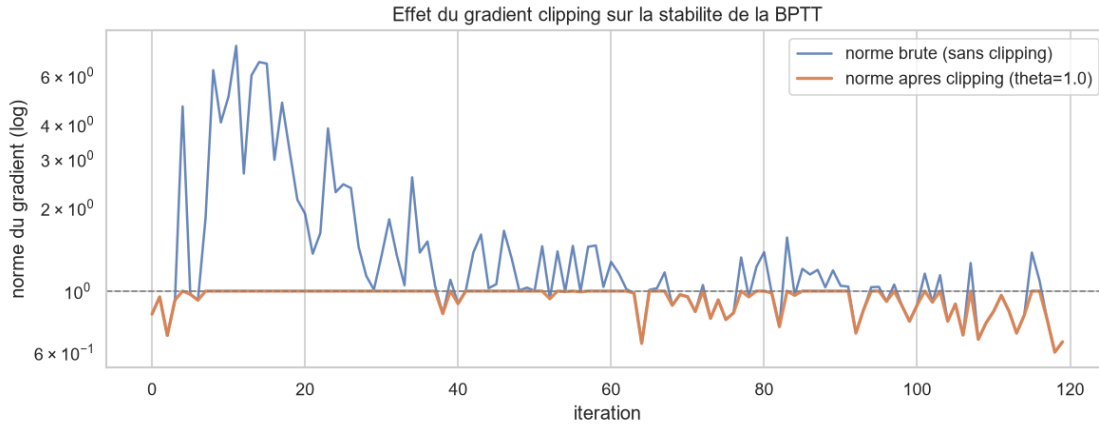


Figure 9 - Effet du gradient clipping sur la stabilité de la BPTT.

Sans clipping, la norme du gradient explose jusqu'à $\approx 7,7$; le clipping la plafonne à $\theta = 1,0$, rendant la descente régulière — illustration directe de son rôle stabilisateur pour les RNN.

6.5 Système Seq2Seq et décodage

Encodeur GRU + décodeur GRU, entraînement par teacher forcing, perte masquée et clipping :

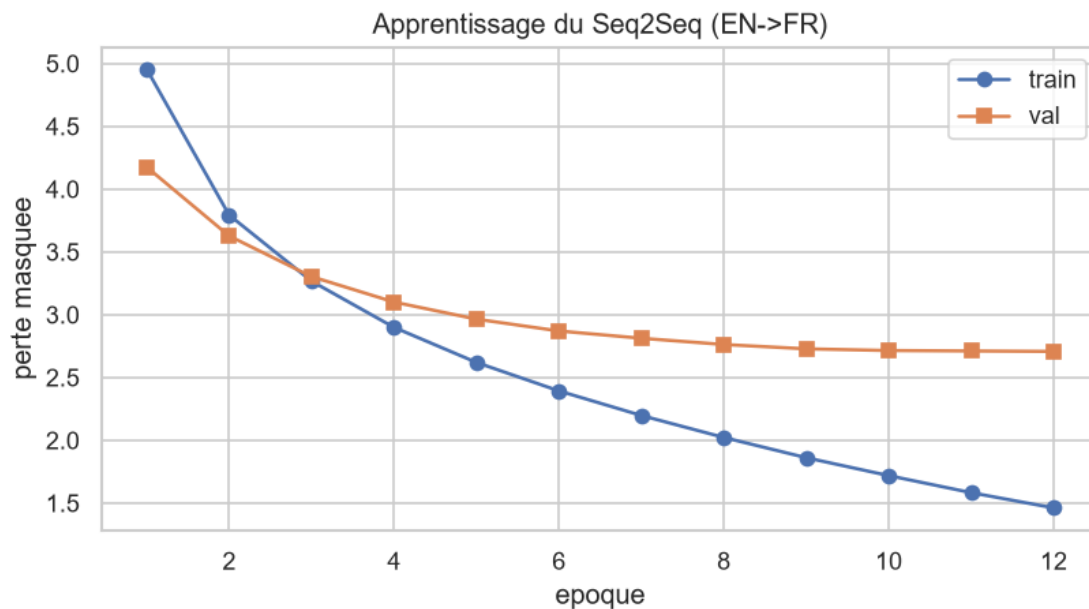


Figure 10 - Apprentissage du système Seq2Seq (EN->FR).

La perte de validation décroît régulièrement (perplexité finale $\approx 15,0$). Deux stratégies de décodage sont comparées par BLEU (implémenté « maison », précision des n-grammes + pénalité de brièveté) :

Décodage	BLEU (val.)
Glouton	0,0155
Beam search (k = 3)	0,0201

Table 5 - Scores BLEU selon la stratégie de décodage.

- Exemples de traductions (le beam corrige souvent le greedy) :

Source EN	Référence FR	Glouton	Beam
he ' s nervous and easily frightened .	il est nerveux et facilement effrayé .	il est un <unk> de <unk> .	il est un peu <unk> .
his debt came to 100 dollars .	sa dette a atteint 100 dollars .	c ' est mon meilleur ami au lycée soir soir tom	c ' était mon meilleur ami .
i ' m very concerned about her illness .	je m ' inquiète beaucoup de sa maladie .	je ne suis pas si je suis fatigué .	je ne suis pas si je <unk> .

Table 6 - Exemples de traductions (glouton vs beam search).

6.6 Interprétation et analyse critique

Les architectures récurrentes battent largement les n-grammes (perplexité < 31). Le GRU est le meilleur compromis ; le gradient clipping est indispensable à la stabilité. Le beam

search améliore le BLEU (0,020 vs 0,016) et produit des phrases plus grammaticales, au prix d'un coût accru. Les traductions restent imparfaites (beaucoup de <unk>, contexte compressé en un seul vecteur) : c'est attendu au vu du budget réduit et de l'absence d'attention.

Question de synthèse — Partie III. *Dans quelle mesure les architectures récurrentes modélisent-elles une séquence, et comment justifier le passage RNN → LSTM/GRU → encodeur-décodeur ?* — Les RNN factorisent la séquence par la règle de chaîne et mémorisent le passé dans un état caché, gérant longueurs variables et ordre. Le RNN simple souffre de la BPTT (gradients instables) ; le passage au LSTM/GRU se justifie par leurs portes offrant un chemin de gradient stable — le GRU le montre par sa perplexité. Le schéma encodeur-décodeur devient nécessaire quand entrée et sortie sont deux séquences non alignées (traduction) ; sa limite (compression du contexte) motive l'introduction ultérieure de l'attention.

7. Synthèse transversale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle ?

Les trois parties partagent exactement le même paradigme (modèle paramétré, perte différentiable, rétropropagation, découpage train/val/test). Ce qui change est le biais inductif :

Critère	MLP (Partie I)	CNN (Partie II)	RNN/Seq2Seq (Partie III)
Structure exploitée	aucune (attributs symétriques)	localité spatiale, invariance	ordre, temporalité
Mécanisme clé	couches denses	partage de poids + champs locaux	état caché récurrent
Brique	nn.Linear	nn.Conv2d	nn.RNN/LSTM/GRU
Résultat clé	F1 0,976	CNN > MLP à ½ paramètres	GRU meilleure PPL ; clipping stabilise

Table 7 - Biais inductifs des trois familles d'architectures.

Adapter l'architecture à la donnée, c'est encoder la bonne hypothèse a priori : symétrie des attributs pour le tabulaire, localité/invariance pour l'image, mémoire pour la séquence. Le même apprentissage supervisé doit donc être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données — ce que nos trois études valident de bout en bout.

8.Limites

- Budget de calcul. Exécution CPU : sous-échantillonnage (Fashion-MNIST, fra-eng) et peu d'époques. Les scores absolus (notamment le BLEU) sont en deçà de l'état de l'art ; augmenter Config (données, époques, dimensions) comblerait l'essentiel de l'écart.
- Partie I. Dataset petit et quasi séparable : peu discriminant entre architectures ; comparaison à des modèles à arbres non incluse.
- Partie II. Pas d'augmentation de données ni d'architecture profonde (ResNet) ; confusions inhérentes entre classes vestimentaires proches.
- Partie III. Pas de mécanisme d'attention (goulot d'étranglement du contexte) ; vocabulaire restreint → nombreux <unk> ; LSTM sous-exploité au budget choisi.

9.Conclusion

Ce projet a couvert, sous PyTorch, les trois grandes familles d'architectures du module et démontré expérimentalement que le choix de l'architecture doit suivre la structure des données. Le MLP excelle sur le tabulaire séparable (F1 0,976) ; le CNN surpasse le MLP sur l'image avec moins de paramètres (0,874 vs 0,850) grâce à la localité et au partage des poids ; les modèles récurrents modélisent efficacement le texte, le GRU offrant le meilleur compromis et le *gradient clipping* garantissant la stabilité, tandis que le Seq2Seq réalise une traduction conditionnelle améliorée par le *beam search*. Au-delà des scores, l'enseignement central est conceptuel : un même paradigme d'apprentissage supervisé se décline en biais inductifs distincts — géométrie, dépendance locale, temporalité — ce qui constitue le cœur de l'ingénierie en deep learning.

Livrables associés : notebooks exécutables notebooks/01..04, code commenté, figures artifacts/figures/, métriques artifacts/figures/results.json.